Week 11 (CS 418 @ UIC)

Week 11 Slide Deck

Recommendation Systems; A/B Testing

Jack Bandy 2026

Topic Title Placeholder

CS 418 · Week 11 · 🟠 35th/Archer 🟠


35/Archer
35/Archer
35/Archer
35/Archer
Orange Line

Demo Content Slide

Placeholder content for Week 11.

- Recommendation systems
- A/B testing
- Evaluation metrics

Examples of Recommender Systems

Amazon: “Customers Who Bought This…”

- Popularized **item-to-item collaborative filtering**: recommend items similar to what you bought, where “similar” means *bought by the same people*[1]
- Works from behavior (purchases, views) — no ratings required
- The *Touching the Void* story is this system finding the long tail

1. Linden, Smith & York, “Amazon.com recommendations: item-to-item collaborative filtering,” *IEEE Internet*

Feeds

- **TikTok / Shorts / Reels** — feed algorithms are essentially recommenders

And Also…

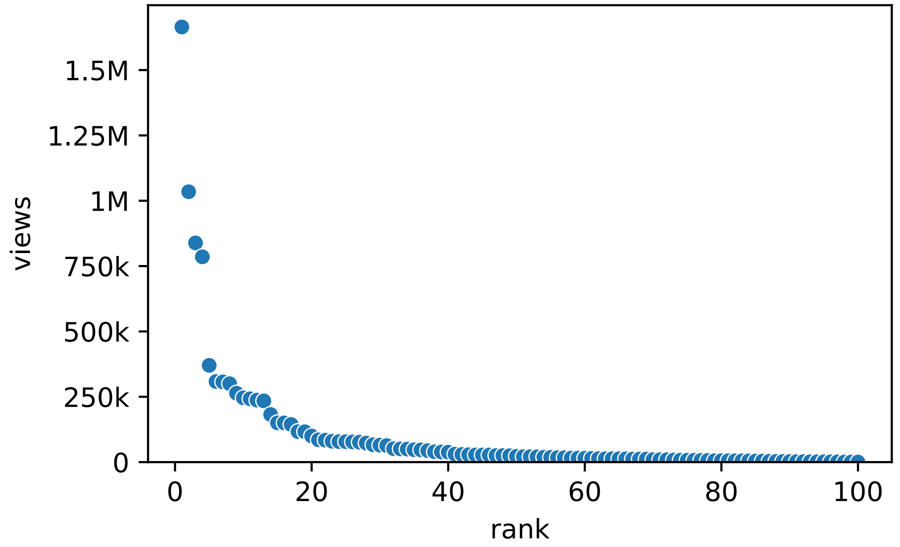
- **Spotify** — E.g. Discover Weekly, collaborative filtering over listening histories
- **Online dating, job boards, app stores** — same basic matrix structure, different items
- **News feeds** — articles similar to what you've read, or read by similar readers

The Premise

- Suppose we pulled 100 random movies from IMDB/Letterboxd
- Sort them by popularity
- Views/popularity on the y axis
- “Hits” appear on the left side

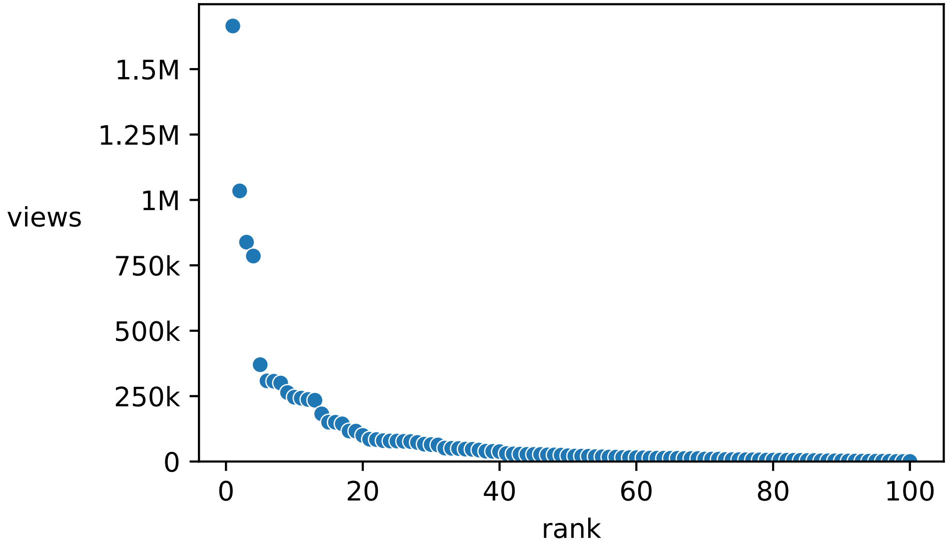
The Premise, Plotted: Synthetic Data

```python
import numpy as np, polars as pl, seaborn as sns
from matplotlib.ticker import EngFormatter
views = np.sort(np.random.default_rng(418).lognormal(10, 2, 100))[::-1]
movies = pl.DataFrame({"rank": np.arange(1, 101), "views": views})
ax = sns.scatterplot(movies, x="rank", y="views")
ax.set_ylim(bottom=0)
ax.yaxis.set_major_formatter(EngFormatter(sep=""))
```



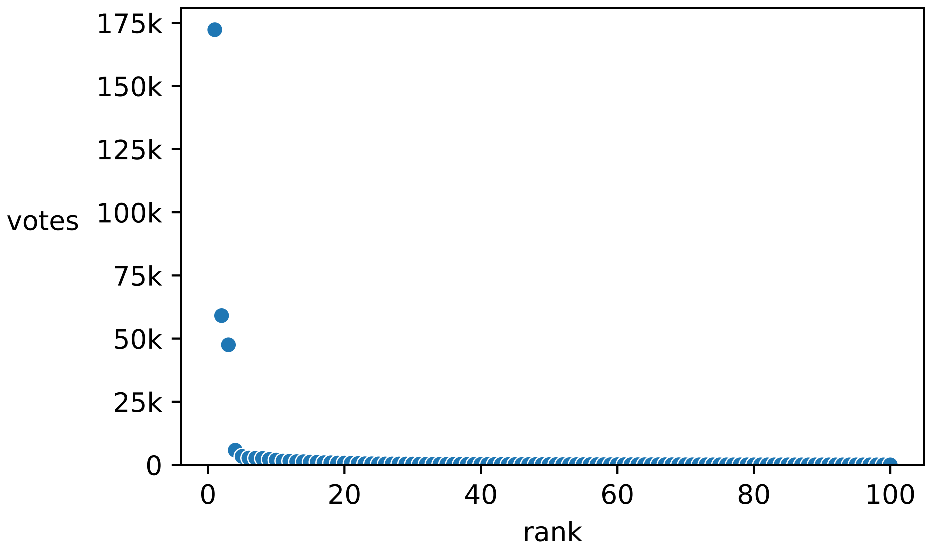
The Premise, Plotted: Synthetic Data

```python
import numpy as np, polars as pl, seaborn as sns
from matplotlib.ticker import EngFormatter
views = np.sort(np.random.default_rng(418).lognormal(10, 2, 100))[::-1]
movies = pl.DataFrame({"rank": np.arange(1, 101), "views": views})
ax = sns.scatterplot(movies, x="rank", y="views")
ax.set_ylim(bottom=0)
ax.set_ylabel("views", rotation=0, labelpad=20)
ax.yaxis.set_major_formatter(EngFormatter(sep=""))
```



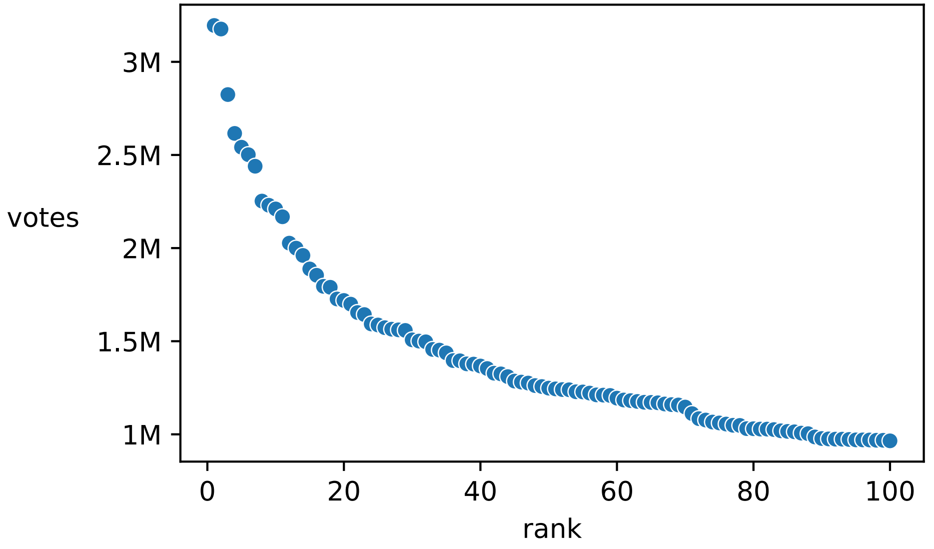
The Premise, Plotted: 100 Random Movies

```python
imdb = pl.read_csv("../../datasets/movies-from-imdb/random.csv").sample(100, seed=418)
imdb = imdb.sort("numVotes", descending=True).with_row_index("rank", offset=1)  # rank from 1, not 0
ax = sns.scatterplot(imdb, x="rank", y="numVotes")
ax.set_ylim(bottom=0)
ax.set_ylabel("votes", rotation=0, labelpad=20)
ax.yaxis.set_major_formatter(EngFormatter(sep=""))
```



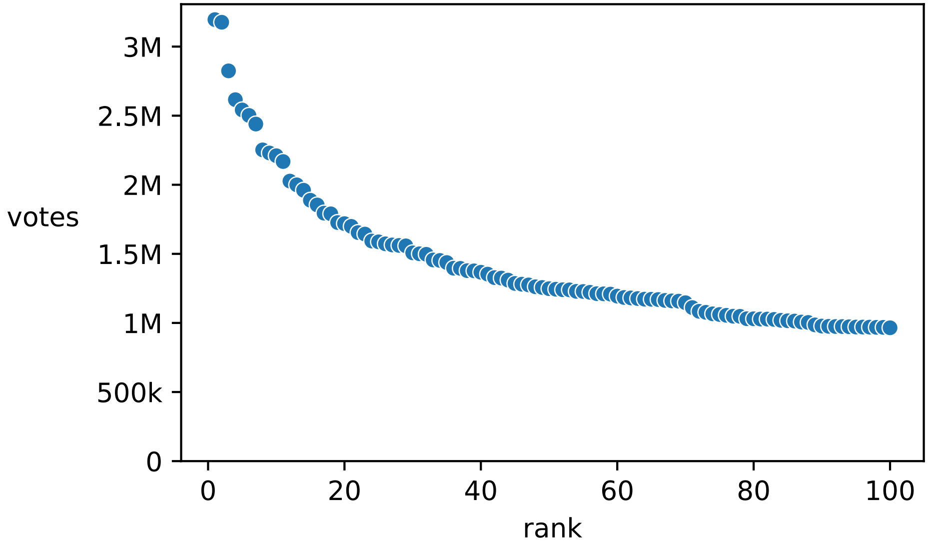
The Premise, Plotted: 100 Most Popular Movies

```python
imdb = pl.read_csv("../../datasets/movies-from-imdb/popular.csv").head(100)
imdb = imdb.with_row_index("rank", offset=1)  # rank from 1, not 0
ax = sns.scatterplot(imdb, x="rank", y="numVotes")
ax.set_ylabel("votes", rotation=0, labelpad=20)
ax.yaxis.set_major_formatter(EngFormatter(sep=""))
```



The Premise, Plotted: 100 Most Popular Movies

```python
imdb = pl.read_csv("../../datasets/movies-from-imdb/popular.csv").head(100)
imdb = imdb.with_row_index("rank", offset=1)  # rank from 1, not 0
ax = sns.scatterplot(imdb, x="rank", y="numVotes")
ax.set_ylim(bottom=0)
ax.set_ylabel("votes", rotation=0, labelpad=20)
ax.yaxis.set_major_formatter(EngFormatter(sep=""))
```



How Do Recommender Systems Work?

Basic Architectures

1. **Content-based**: recommend items whose *properties* are similar to items you liked
    - “You liked three movies with Julia Roberts…”
    - “You liked action movies…”
    - “You liked movies <90mins…”
2. **Collaborative filtering**: recommend items liked by *users similar to you*
    - “People who watched X also watched Y…”

Example: What YouTube Knows

For every (user, video) pair, YouTube can record a **utility**:

	lofi study mix	cat fails	intro to SQL	marathon vlog	sourdough 101
Alice	95%		80%		10%
Bob		100%		60%	
Carmen	90%	15%			
Dev			85%	70%	

Consider utility as **watch percentage** (blank = never clicked/watched)

Sources

1. GitHub source: https://github.com/jackbandy/data-science-fun/blob/main/docs/slides/week11.qmd.
2. Last modified and compiled June 24, 15:01h Central (Chicago) time.
3. Recommendation systems content follows Leskovec, Rajaraman & Ullman, *Mining of Massive Datasets*, Chapter 9.
4. Linden, Smith & York, “Amazon.com recommendations: item-to-item collaborative filtering,” *IEEE Internet Computing* 7:1 (2003). https://doi.org/10.1109/MIC.2003.1167344
5. Slides developed using materials from Elena Zheleva and Gonzalo Bello Lander, the Berkeley DS 100 team, Marine Carpuat, and Brian Ziebart.
6. Slide deck built with Quarto revealjs.
7. Title font is Big Shoulders; Body font is Libre Franklin.